

Séance 4 — Exercice : sécuriser et tester le module Réservation

KFOKAM48 — Batch 2 — Phase 3 • Séances Full Stack
Backend — Spring Security et tests

Le contexte

Votre API de réservation fonctionne, mais **elle est ouverte à tous**. N'importe qui peut aujourd'hui consulter les réservations d'un autre adhérent, ou annuler celles qu'il n'a pas faites.

Vous allez la fermer, puis prouver par des tests que vos règles tiennent.

Aucun code ne vous sera donné. La documentation Spring Security et le projet existant sont vos sources.

Partie 1 — La sécurité

Les rôles

Deux rôles : **ADHERENT** et **BIBLIOTHECAIRE**.

Si votre projet possède déjà un système de rôles, appuyez-vous dessus. Sinon, créez-le.

Les autorisations à mettre en place

Endpoint	Anonyme	ADHERENT	BIBLIOTHECAIRE
POST /api/reservations	NON	OUI pour lui-même uniquement	OUI pour n'importe qui
GET /api/reservations	NON	OUI ses réservations seulement	OUI toutes
GET /api/reservations/{id}	NON	OUI si elle lui appartient	OUI toutes
PATCH /api/reservations/{id}/annuler	NON	OUI si elle lui appartient	OUI toutes
DELETE /api/reservations/{id}	NON	NON	OUI

Les règles de sécurité

Réf.	Règle
RS-01	Sans token, tout endpoint de réservation renvoie 401
RS-02	Un ADHERENT qui tente une action réservée au bibliothécaire reçoit 403
RS-03	Un ADHERENT qui accède à la réservation d'un autre reçoit 403
RS-04	Un ADHERENT ne peut pas créer une réservation au nom d'un autre adhérent
RS-05	Un GET /api/reservations par un ADHERENT ne retourne que ses propres réservations

Attention à RS-04. Le client envoie `adherentId` dans le corps de la requête. Un adhérent malveillant peut y mettre l'identifiant de quelqu'un d'autre. **L'identité doit venir du token, pas du corps de la requête.**

Distinguer 401 et 403

Code	Signification
401 Unauthorized	Je ne sais pas qui vous êtes — token absent, invalide ou expiré
403 Forbidden	Je sais qui vous êtes, mais vous n'avez pas le droit

Renvoyer un **403** à un utilisateur non authentifié, ou un **401** à un utilisateur authentifié sans droits, est une erreur.

Partie 2 — Les tests

Test unitaire — la règle RG-03

Écrivez un test unitaire sur la couche service qui vérifie la limite de 3 réservations actives.

Le repository doit être **simulé** (mock), pas connecté à une vraie base. Le test doit passer sans qu'aucune base ne tourne.

Deux cas à couvrir :

- un adhérent ayant 2 réservations actives peut en créer une troisième
- un adhérent ayant 3 réservations actives reçoit un refus

Test d'intégration — un endpoint sécurisé

Écrivez un test d'intégration sur `GET /api/reservations` qui vérifie :

- sans token -> **401**
- avec un token `ADHERENT` -> **200**
- avec un token `ADHERENT`, sur la réservation d'un autre -> **403**

Exigences

- Les tests s'exécutent par la commande de test du projet, sans manipulation manuelle
- Ils passent en vert
- Les noms de méthodes décrivent ce qui est testé, pas `test1` ou `testService`

Barème

Élément	Points
Authentification exigée sur tous les endpoints (RS-01)	4
Autorisations par rôle correctement appliquées (RS-02)	5
Un adhérent ne voit et ne modifie que ses réservations (RS-03, RS-05)	5
L'identité vient du token, pas du corps de la requête (RS-04)	4
Distinction correcte entre 401 et 403	2

Élément	Points
Test unitaire sur RG-03, avec repository simulé	5
Test d'intégration sur un endpoint sécurisé	5
Total	30

RS-04 est le poste le plus important de la partie sécurité. Une API qui fait confiance au corps de la requête pour savoir qui parle n'est pas sécurisée, même si tous les autres contrôles sont en place.

Le passage devant le formateur

Vous présenterez votre travail en **8 minutes**, en le manipulant vous-même.

À préparer avant : deux comptes **ADHERENT** distincts et un compte **BIBLIOTHECAIRE**, avec leurs identifiants notés. Chaque adhérent doit avoir au moins une réservation à son nom.

Attendez-vous à devoir provoquer des refus en direct, et à lancer vos tests devant le formateur.

Livrable

Une branche `feature/reservation-securite-prenom-nom`, une Pull Request décrite, relue par un pair.

Dans la description : la capture du résultat de vos tests, et une phrase par règle **RS - 01** à **RS - 05** indiquant où vous l'avez implémentée.

Si vous finissez en avance

- Gérer l'expiration du token et le message associé
- Ajouter un test sur RG-01 (réservation d'un livre disponible)
- Journaliser les tentatives d'accès refusées